



UNIVERSIDAD INTERNACIONAL DEL ECUADOR

Facultad de Sistemas de Información
Carrera de Ingeniería en Ciberseguridad

PROYECTO INTEGRADOR

*El impacto de las nuevas tecnologías en la sociedad:
desarrollo y proyección de soluciones informáticas*

BASTION

Bóveda Personal de Credenciales

Estudiante David Alexander Benz Zambrano

Correo institucional dabezza@uide.edu.ec

Asignatura Lógica de Programación

Docente Msc. Lilian Aman

Paralelo 1-CIB-1A

Quito - Ecuador - junio de 2026

Índice de contenidos

Resumen

1. Introducción

2. Descripción del problema

3. Relación con los contenidos de la asignatura

4. Explicación del sistema desarrollado

5. Reflexión sobre el impacto de las nuevas tecnologías en la sociedad

6. Cronograma de actividades

7. Conclusiones, implicaciones y limitaciones

Referencias

Resumen

BASTION es una bóveda personal de credenciales de línea de comandos desarrollada en Python que aborda un problema cotidiano de la sociedad digital: la dispersión y la reutilización de contraseñas débiles frente a la creciente cantidad de servicios en línea. El sistema genera contraseñas seguras de forma automática y las almacena en un archivo local protegido por una contraseña maestra, cuyo hash se resguarda mediante el algoritmo SHA-256. Al operar sin dependencia de la nube, reduce la superficie de exposición y mantiene el control de los datos en el propio equipo del usuario. La aplicación se estructura en tres capas (presentación, lógica de negocio y persistencia) e implementa estructuras repetitivas y de selección para la autenticación controlada por intentos, la generación de claves y la búsqueda de credenciales. El diseño se fundamenta en estándares reconocidos de identidad, aleatoriedad y almacenamiento seguro de contraseñas, y se contextualiza en una agencia tecnológica ecuatoriana, ofreciendo una solución informática concreta ante los retos que imponen las nuevas tecnologías.

Palabras clave: gestión de credenciales, contraseña maestra, SHA-256, seguridad de la información, programación en Python, autenticación.

1. Introducción

La expansión acelerada de las nuevas tecnologías ha multiplicado el número de servicios digitales que una persona utiliza a diario, desde correos electrónicos y redes sociales hasta plataformas bancarias, sistemas de trabajo y accesos técnicos a servidores. Cada uno de estos servicios exige credenciales propias, lo que ha convertido la gestión de contraseñas en un desafío cotidiano para los usuarios. En la práctica, la sobrecarga de claves favorece conductas inseguras como la reutilización de una misma contraseña en múltiples cuentas o la elección de combinaciones fáciles de recordar pero igualmente fáciles de vulnerar. Esta tensión entre comodidad y seguridad refleja con claridad el tema del proyecto integrador, "El impacto de las nuevas tecnologías en la sociedad: desarrollo y proyección de soluciones informáticas", pues evidencia que el avance tecnológico no solo genera oportunidades, sino también riesgos que demandan respuestas técnicas responsables.

En este escenario surge BASTION, una bóveda personal de credenciales de línea de comandos desarrollada en Python como respuesta concreta a dicha problemática. El sistema permite generar contraseñas robustas, almacenarlas de manera organizada por categorías y recuperarlas mediante una única contraseña maestra que el usuario debe recordar. A diferencia de los gestores que residen en la nube, BASTION conserva la información en un archivo local del propio equipo, lo que reduce la superficie de ataque y mantiene el control de los datos en manos del usuario. La protección de la contraseña maestra se realiza a través de su hash con el algoritmo SHA-256, de modo que el valor en claro nunca se almacena. Como buena práctica de seguridad, las directrices de identidad digital recomiendan no transmitir ni guardar secretos en texto plano y verificar las credenciales mediante funciones resistentes (Grassi et al., 2017), criterio que orienta el diseño del producto.

El proyecto se sustenta en estándares reconocidos en el ámbito de la seguridad de la información. La gestión de la identidad y la autenticación se inspira en las recomendaciones del NIST sobre verificación de credenciales (Grassi et al., 2017); la generación de claves considera los principios de aleatoriedad necesarios para producir secretos impredecibles (Eastlake et al., 2005); y el

almacenamiento adecuado de contraseñas se alinea con las prácticas promovidas por la comunidad de seguridad de aplicaciones (OWASP Foundation, 2025). Asimismo, el diseño contempla como evolución del producto la incorporación de cifrado autenticado AES-256-GCM (Dworkin, 2007) y de funciones de derivación de clave robustas, lo que delimita una hoja de ruta clara entre la versión académica actual y una versión reforzada para entornos profesionales. La arquitectura en tres capas y la organización modular del código responden, además, a buenas prácticas de ingeniería de software orientadas a la mantenibilidad (Sommerville, 2016).

El objetivo general del sistema es desarrollar una solución informática de gestión personal de credenciales que centralice y proteja las contraseñas del usuario en un entorno local, aplicando estructuras de programación y fundamentos de seguridad. De este objetivo se derivan los siguientes objetivos específicos: (1) implementar un mecanismo de autenticación mediante contraseña maestra con un número limitado de intentos, controlado por un bucle con contador; (2) desarrollar un generador de contraseñas seguras que construya las claves carácter por carácter y evalúe su nivel de fortaleza; (3) diseñar la persistencia local de las credenciales en un archivo estructurado, resguardando el hash de la contraseña maestra; y (4) ofrecer funcionalidades de organización por categorías, búsqueda sin distinción de mayúsculas y visualización enmascarada de las contraseñas. Con ello, BASTION ilustra cómo un proyecto académico puede proyectarse hacia una necesidad real, contextualizada en Blue Nova Consulting, agencia tecnológica ecuatoriana que requiere centralizar credenciales técnicas dispersas como accesos a servidores, claves de API y llaves SSH.

2. Descripción del problema

En la actualidad, una sola persona administra decenas de cuentas y servicios digitales que exigen credenciales de acceso: redes sociales, banca y finanzas en línea, correo electrónico, herramientas de trabajo y, en el caso de perfiles técnicos, accesos a servidores, claves de interfaces de programación de aplicaciones (API) y llaves SSH. Frente a esta sobrecarga, el comportamiento más frecuente consiste en gestionar las contraseñas de forma insegura mediante tres prácticas recurrentes. La primera es la reutilización de una misma contraseña en múltiples servicios, lo que convierte la filtración de un solo sitio en un compromiso en cadena de todas las cuentas asociadas. La segunda es el uso de claves débiles y predecibles, construidas con palabras del diccionario, nombres, fechas o secuencias sencillas, fáciles de adivinar mediante ataques de fuerza bruta o de diccionario. La tercera es el almacenamiento sin cifrar de las credenciales en archivos de texto plano, hojas de cálculo, notas del navegador o mensajes enviados a uno mismo, donde quedan expuestas ante cualquiera que obtenga acceso al dispositivo.

Estas prácticas son especialmente relevantes en la sociedad digital actual porque la identidad de las personas se sostiene cada vez más sobre la autenticación por contraseña, y un fallo en su gestión compromete simultáneamente la privacidad, el patrimonio y la reputación del usuario. Las directrices de identidad digital del NIST advierten que la fortaleza real de una credencial depende más de su longitud y aleatoriedad que de reglas de composición artificiales, y desaconsejan obligar a esquemas memorizables que terminan empujando a la reutilización (Grassi et al., 2017). En la misma línea, las recomendaciones sobre almacenamiento de contraseñas insisten en que ningún secreto debe conservarse en texto plano, sino protegido mediante funciones criptográficas adecuadas (OWASP Foundation, 2025). La brecha entre lo que recomiendan estos estándares y lo que efectivamente hace una persona sin herramientas de apoyo es, precisamente, el espacio donde se

concentra el riesgo: el usuario sabe que debería emplear claves únicas y robustas, pero carece de un medio práctico para generarlas y recordarlas sin recurrir a atajos inseguros.

BASTION aborda este problema ofreciendo una bóveda personal de credenciales de línea de comandos que separa dos responsabilidades críticas: la generación de contraseñas fuertes y su almacenamiento protegido. Por un lado, el sistema incorpora un generador que construye contraseñas con longitud configurable y combinación de minúsculas, mayúsculas, dígitos y símbolos, junto con un indicador de fortaleza que clasifica cada clave como débil, media o fuerte; de este modo, el usuario deja de depender de su memoria o de patrones predecibles y obtiene credenciales únicas para cada servicio. Por otro lado, toda la información se concentra en una bóveda local en formato JSON, sin envío a la nube, lo que reduce la superficie de ataque al eliminar la exposición en tránsito y la dependencia de servidores de terceros. El acceso a esa bóveda queda condicionado a una única contraseña maestra, cuyo hash se almacena con SHA-256 en lugar del valor en claro, y a un control de inicio de sesión que limita los intentos fallidos y bloquea el acceso tras un máximo de tres, lo que dificulta los ataques por adivinación. Así, el usuario solo necesita recordar una clave robusta para custodiar de forma centralizada todas las demás.

Se eligió este caso entre las opciones de la asignatura porque combina una necesidad real y verificable con un alcance técnico adecuado a las estructuras de programación estudiadas. La gestión segura de credenciales es un problema cotidiano y cercano para el autor, cuya agencia Blue Nova Consulting acumula accesos técnicos dispersos -credenciales de servidores, claves de API y llaves SSH- que conviene centralizar de forma segura, lo que dota al proyecto de un propósito concreto más allá del ejercicio académico. Al mismo tiempo, el problema permite ejercitar de manera natural los contenidos de la materia: bucles while con contador para el control de intentos de inicio de sesión, bucles for con `range()` para la construcción carácter por carácter de cada contraseña, recorridos con `.items()` para la búsqueda y el listado de credenciales, y estructuras de selección if/elif/else para los menús, las validaciones y el cálculo de la fortaleza. Esta correspondencia entre un problema socialmente pertinente y los objetivos de aprendizaje, sumada a la posibilidad de sustentar el diseño en estándares reconocidos del área de ciberseguridad, justifica la selección de BASTION como caso de estudio del presente trabajo.

El desarrollo de BASTION no constituye un ejercicio aislado, sino la aplicación integrada de los contenidos trabajados a lo largo de las cuatro unidades de la asignatura Lógica de Programación. Cada unidad aportó un conjunto de competencias que se materializa de forma directa y verificable en el código fuente del producto (`bastion.py`). A continuación se detalla, unidad por unidad, cómo el proyecto evidencia ese aprendizaje progresivo, desde el análisis inicial del problema hasta la integración funcional del producto terminado.

3.1. Unidad 1 - Fundamentos, análisis del problema y diseño de la solución

La primera unidad sentó las bases del razonamiento algorítmico: comprender un problema, descomponerlo y representar su solución antes de escribir una sola línea de código. BASTION nace precisamente de ese ejercicio de análisis. El problema identificado fue la dispersión de credenciales técnicas (accesos a servidores, claves de API y llaves SSH) en el contexto de Blue Nova Consulting, y la solución se planteó como una bóveda local que centralizara esas credenciales bajo una contraseña maestra.

Sobre esta base se elaboraron los diagramas de funcionalidad solicitados en la unidad. El diagrama de casos de uso recoge las acciones del usuario (crear bóveda, iniciar sesión, generar contraseña, agregar credencial, buscar, ver todas y cambiar la contraseña maestra), que más tarde se tradujeron una a una en funciones del programa, cada una documentada con su identificador de caso de uso (por ejemplo, CU-01 en `crear_boveda` o CU-03 en `generar_contrasena`). El diagrama de flujo describió la lógica de validación y de los bucles antes de implementarlos, y el diagrama de arquitectura definió la separación en tres capas (presentación, lógica de negocio y persistencia). Esa arquitectura no quedó solo en el papel: el código está organizado en bloques comentados que reproducen exactamente esas tres capas, lo que demuestra la trazabilidad entre el diseño de la Unidad 1 y el producto final.

3.2. Unidad 2 - Entorno de desarrollo, control de versiones y tipos de datos

La segunda unidad marcó el inicio de la codificación, la preparación del entorno de trabajo y el uso de GitHub para el control de versiones. El proyecto se desarrolló en un entorno de Python utilizando únicamente la biblioteca estándar (`hashlib`, `json`, `os`, `secrets`, `string`, `getpass` y `sys`), decisión coherente con el carácter académico del trabajo, y se gestionó mediante un repositorio que incluye archivos propios de un proyecto versionado, como `README.md`, `LICENSE` y un archivo de exclusiones para no subir la bóveda local al historial.

Los conceptos de variables y tipos de datos primitivos se aplican de manera constante. Se emplean cadenas de texto (`str`) para los nombres de servicio, los usuarios y las contraseñas; enteros (`int`) para la longitud de la contraseña y para el contador de intentos; y valores booleanos (`bool`) para las decisiones del usuario, como `incluir_mayusculas`, `incluir_numeros` e `incluir_simbolos`, que determinan qué conjuntos de caracteres se incorporan. También se aprovecha la conversión y validación de tipos: la función `pedir_entero` verifica con `isdigit()` que la entrada sea numérica antes de convertirla con `int()`, evitando errores en tiempo de ejecución. El manejo correcto de tipos resulta especialmente relevante en seguridad, donde la contraseña maestra se codifica a bytes en UTF-8 antes de calcular su resumen criptográfico, tal como exige el procesamiento de funciones hash (Stallings, 2023).

3.3. Unidad 3 - Estructuras de decisión, bucles, contadores y operadores

La tercera unidad constituye el núcleo lógico de BASTION, pues las estructuras de control son las que dan vida a su comportamiento. El proyecto implementa de forma explícita y comentada los bucles repetitivos, las estructuras de selección, los contadores y los operadores estudiados.

Bucles while

El bucle `while` se utiliza con dos finalidades. La primera es la validación de entradas: funciones como `preguntar_si_no`, `pedir_entero`, `crear_boveda` y `cambiar_contrasena_maestra` emplean `while True` para insistir hasta que el usuario proporcione un dato correcto, saliendo del ciclo con `break` solo cuando la validación es exitosa. La segunda, y de mayor valor desde la perspectiva de la ciberseguridad, es el control de intentos en el inicio de sesión. La función `iniciar_sesion` declara

un contador `intentos = 0` y ejecuta `while intentos < MAX_INTENTOS`, permitiendo un máximo de tres intentos. Si la contraseña es correcta se devuelve la bóveda y se rompe el ciclo; si es incorrecta, el contador aumenta y, al agotarse los intentos, el programa bloquea el acceso y se cierra con `sys.exit()`. Este mecanismo aplica de manera práctica la recomendación de limitar los intentos de autenticación para mitigar ataques de fuerza bruta (Grassi et al., 2017).

Bucles for

El bucle `for` aparece en dos variantes. La primera, `for` con `range()`, se utiliza en `generar_contrasena` mediante la instrucción `for i in range(longitud_deseada)`, que se repite tantas veces como caracteres tenga la contraseña deseada y, en cada iteración, agrega un carácter elegido al azar del alfabeto construido (`contrasena_generada += secrets.choice(alfabeto)`). La segunda variante recorre estructuras de datos: en `buscar_credencial` y en `ver_credenciales` se emplea `for servicio, datos in credenciales.items()` para iterar el diccionario de credenciales obteniendo a la vez la clave (el servicio) y su valor (los datos asociados). Adicionalmente, en `agregar_credencial` se recorre el diccionario de categorías con un `for` para presentar el menú numerado al usuario.

Estructuras if / elif / else

Las estructuras de selección gobiernan tanto los menús como las reglas de negocio. Las funciones `main` y `menu_principal` emplean cadenas `if / elif / else` para enrutar la opción elegida hacia la acción correspondiente y para advertir cuando la opción no es válida. La selección también sostiene una regla de seguridad: el indicador de fortaleza se calcula en `calcular_fortaleza` con un `if / elif / else` que clasifica la contraseña como Débil, Media o Fuerte según su longitud y la cantidad de tipos de caracteres empleados.

Contadores

Los contadores se aplican con el patrón de incremento `+= 1`. Además del contador de `intentos` ya descrito, la variable `tipos_seleccionados` cuenta cuántos conjuntos de caracteres se incluyen en la contraseña (partiendo de 1 por las minúsculas y aumentando por cada tipo añadido), valor que luego alimenta el cálculo de fortaleza. Asimismo, la variable `total` en `ver_credenciales` contabiliza cuántas credenciales se muestran, para informar el total al final del listado.

Operadores relacionales y lógicos

Los operadores son la condición que activa cada decisión. Se usan operadores relacionales para comparar longitudes y rangos (`len(clave) < LONGITUD_MINIMA`, `numero < minimo` or `numero > maximo`, `longitud <= 11`) y para verificar la coincidencia exacta de los hashes en la autenticación (`calcular_hash(clave) == boveda["hash_maestra"]`). Los operadores lógicos combinan condiciones: `or` admite varias formas de responder afirmativamente, `and` exige longitud y diversidad de caracteres para calificar una contraseña como Fuerte (`longitud >= 12` and `tipos_seleccionados >= 3`), y `not` evalúa la bandera de búsqueda (`if not encontrado`). También se emplea el operador de pertenencia `in` para comprobar si un servicio ya existe o si una opción de categoría es válida. Por último, el código se documenta con comentarios descriptivos en cada bloque, según la buena práctica de mantener el código legible y mantenible (Sommerville, 2016).

3.4. Unidad 4 - Programación funcional, estructuras de datos y funciones

La cuarta unidad orientó el trabajo hacia la organización del código en funciones, el uso de estructuras de datos y la evaluación, optimización e integración del producto final. BASTION refleja estos contenidos en su organización modular: el programa se compone de funciones especializadas y reutilizables, cada una con un único propósito, parámetros explícitos y valor de retorno documentado. Por ejemplo, `calcular_hash` recibe un texto y devuelve su resumen, y `enmascarar` recibe una contraseña y devuelve su versión oculta. Esta descomposición funcional evita la repetición de código, facilita las pruebas y se alinea con los principios de modularidad y bajo acoplamiento de la ingeniería de software (Pressman y Maxim, 2021).

En cuanto a las estructuras de datos, BASTION integra los tres tipos estudiados en la unidad: tuplas, listas y diccionarios. Las categorías permitidas se definen en una tupla inmutable, `CATEGORIAS`, porque son valores fijos que no deben modificarse durante la ejecución; el menú de categorías la recorre con `enumerate`. El diccionario es la pieza central del almacenamiento: la bóveda completa es un diccionario que contiene el hash de la contraseña maestra, un subdiccionario de credenciales (donde cada credencial usa el nombre del servicio como clave y, como valor, otro diccionario con su categoría, usuario y contraseña) y una lista de historial. Esta estructura anidada permite el acceso directo por clave y se persiste íntegramente en formato JSON, lo que demuestra la correspondencia natural entre los diccionarios de Python y el formato de intercambio de datos.

Las listas se aplican en el historial de accesos: la bóveda incluye una lista `historial` en la que, mediante la función `registrar_acceso`, se agrega un registro con la fecha y la hora cada vez que el usuario inicia sesión, y que luego se recorre con un bucle `for` y `enumerate` para mostrarlo. Las funciones, además, emplean parámetros por defecto, otro contenido de la unidad: `registrar_acceso(boveda, evento="Acceso concedido")` y `pedir_entero(mensaje, minimo=LONGITUD_MINIMA, maximo=LONGITUD_MAXIMA)` definen valores predeterminados que simplifican su invocación. Esta organización en funciones de propósito único, junto con la integración planificada de cifrado autenticado AES-256-GCM y derivación de clave con Argon2id, evidencia la fase de evaluación y optimización de la unidad, en la que el producto se integra como un todo funcional y se identifican las líneas de mejora continua.

3.5. Tabla resumen

Unidad	Contenido principal	Dónde se aplica en BASTION
Unidad 1	Fundamentos, análisis del problema y diagramas (casos de uso, flujo y arquitectura)	Definición del problema de credenciales dispersas; casos de uso CU-01 a CU-07; arquitectura en tres capas reflejada en la organización del código
Unidad 2	Entorno de desarrollo, control de versiones y tipos de datos (str, int, bool)	Proyecto en Python con biblioteca estándar; repositorio con README, licencia y exclusiones; variables <code>str</code> , <code>int</code> y <code>bool</code> ; validación y conversión de tipos en <code>pedir_entero</code>
Unidad 3	Estructuras de decisión, bucles, contadores y operadores	<code>while</code> con contador de intentos en <code>iniciar_sesion</code> ; <code>for</code> con <code>range()</code> en <code>generar_contrasena</code> ; <code>for</code> con <code>.items()</code> en <code>buscar_credencial</code> y <code>ver_credenciales</code> ; <code>if/elif/else</code> en menús y fortaleza; contadores <code>+= 1</code> ; operadores relacionales, lógicos y de pertenencia
Unidad 4	Programación funcional, estructuras de datos y funciones	Funciones con parámetros, retorno y valores por defecto (<code>registrar_acceso</code> , <code>pedir_entero</code>); tupla <code>CATEGORIAS</code> ; lista <code>historial</code> de accesos; diccionarios de la bóveda; persistencia en JSON

El sistema desarrollado, denominado BASTION, es una bóveda personal de credenciales de línea de comandos (CLI) escrita en Python. Su propósito es centralizar de forma local las credenciales técnicas dispersas de Blue Nova Consulting (accesos a servidores, claves de API y llaves SSH), generando contraseñas seguras y resguardándolas bajo una contraseña maestra. La versión académica se apoya únicamente en la biblioteca estándar de Python (`hashlib`, `json`, `os`, `secrets`, `string`, `getpass` y `sys`), de modo que el programa es portable y no depende de servicios externos ni de la nube, lo que reduce la superficie de ataque.

4.1. Arquitectura en tres capas



Figura 1. Arquitectura en tres capas de BASTION.

El sistema se organiza en una arquitectura de tres capas, patrón clásico de separación de responsabilidades que facilita el mantenimiento y la comprensión del código (Sommerville, 2016). Cada capa tiene una función delimitada y se comunica con las contiguas mediante llamadas a funciones:

Capa de presentación

Es la interfaz de usuario en consola. Está compuesta por los menús de texto y la lectura de opciones del usuario. La concentran las funciones `main()`, que despliega el menú inicial (crear bóveda, iniciar sesión, salir), y `menu_principal()`, que muestra el menú de operaciones una vez la sesión está abierta. Ambas funciones se construyen sobre un bucle `while True` con `break` y una estructura `if / elif / else` que enruta cada elección del usuario hacia la función correspondiente de la capa inferior. A esta capa pertenecen también las utilidades de entrada y validación como `leer_clave_oculta()`, `preguntar_si_no()` y `pedir_entero()`.

Capa de lógica de negocio

Contiene las reglas propias de la aplicación: el generador de contraseñas, las validaciones, el cálculo de fortaleza y los casos de uso. Aquí residen `generar_contrasena()`, `agregar_credencial()`, `buscar_credencial()`, `ver_credenciales()`, `cambiar_contrasena_maestra()`, `iniciar_sesion()` y `calcular_fortaleza()`. Esta capa no interactúa directamente con el usuario ni con el disco: recibe datos ya validados de la capa de presentación y solicita operaciones de lectura y escritura a la capa de persistencia.

Capa de persistencia

Se encarga de leer y escribir la bóveda en el archivo JSON local. La integran las funciones `cargar_boveda()`, `guardar_boveda()`, `existe_boveda()` y la función criptográfica de apoyo `calcular_hash()`. Es la única capa que toca el sistema de archivos, lo que aísla el almacenamiento del resto del programa.

Esta separación concuerda con la recomendación de construir software como un conjunto de componentes con responsabilidades únicas y bajo acoplamiento, de modo que un cambio en el formato de almacenamiento (por ejemplo, migrar de JSON a una base de datos cifrada) afecte solo a la capa de persistencia sin alterar las demás (Pressman y Maxim, 2021).

4.2. Funcionalidades y casos de uso

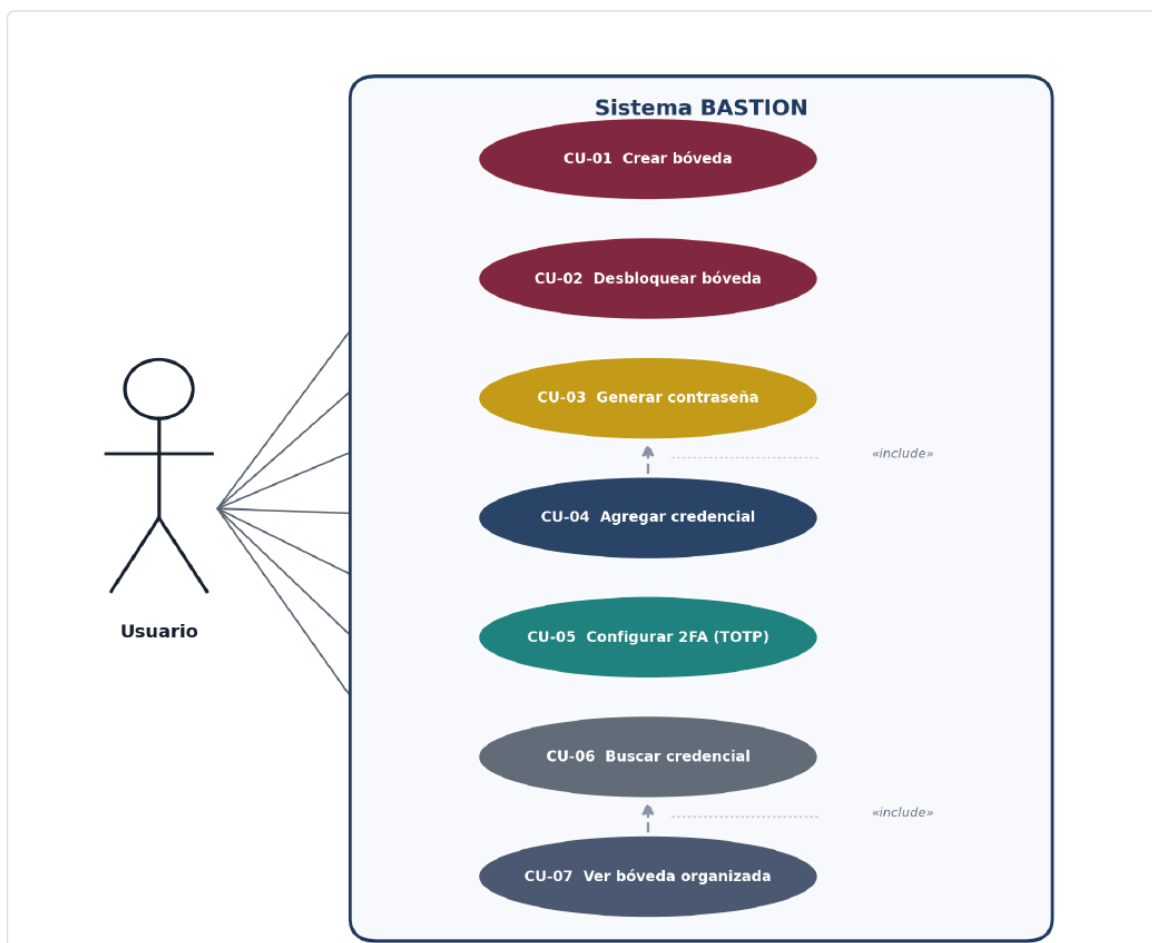


Figura 2. Diagrama de casos de uso del sistema BASTION (UML).

El sistema implementa siete casos de uso identificados desde el diseño previo, accesibles a través de los menús:

Caso de uso	Función responsable	Descripción
CU-01 Crear bóveda	<code>crear_boveda()</code>	Define la contraseña maestra (mínimo 8 caracteres), la confirma, calcula su hash SHA-256 y genera el archivo de bóveda con credenciales vacías.
CU-02 Iniciar sesión (desbloquear)	<code>iniciar_sesion()</code>	Valida la contraseña maestra contra el hash guardado, con máximo de tres intentos y bloqueo posterior.
CU-03 Generar contraseña segura	<code>generar_contrasena()</code>	Construye una contraseña aleatoria según la longitud y los tipos de caracteres elegidos, y estima su fortaleza.
CU-04 Agregar credencial	<code>agregar_credencial()</code>	Registra un servicio con su categoría, usuario y contraseña, y lo persiste.
CU-06 Buscar credencial	<code>buscar_credencial()</code>	Localiza una credencial por nombre del servicio sin distinguir mayúsculas de minúsculas.
CU-07 Ver todas las credenciales	<code>ver_credenciales()</code>	Lista todas las credenciales con la contraseña enmascarada.
CU adicional: Cambiar maestra	<code>cambiar_contrasena_maestra()</code>	Valida la clave actual y la sustituye por una nueva confirmada.

Las credenciales se clasifican en seis categorías predefinidas (Redes sociales, Banca y finanzas, Trabajo, Correo, Claves API / SSH y Otros) y la fortaleza de cada contraseña se reporta en tres niveles: Débil, Media o Fuerte. Una relación de inclusión enlaza CU-03 y CU-04: tras generar una contraseña, el programa ofrece guardarla directamente como credencial, reutilizando el valor producido.

4.3. Estructuras lógicas implementadas

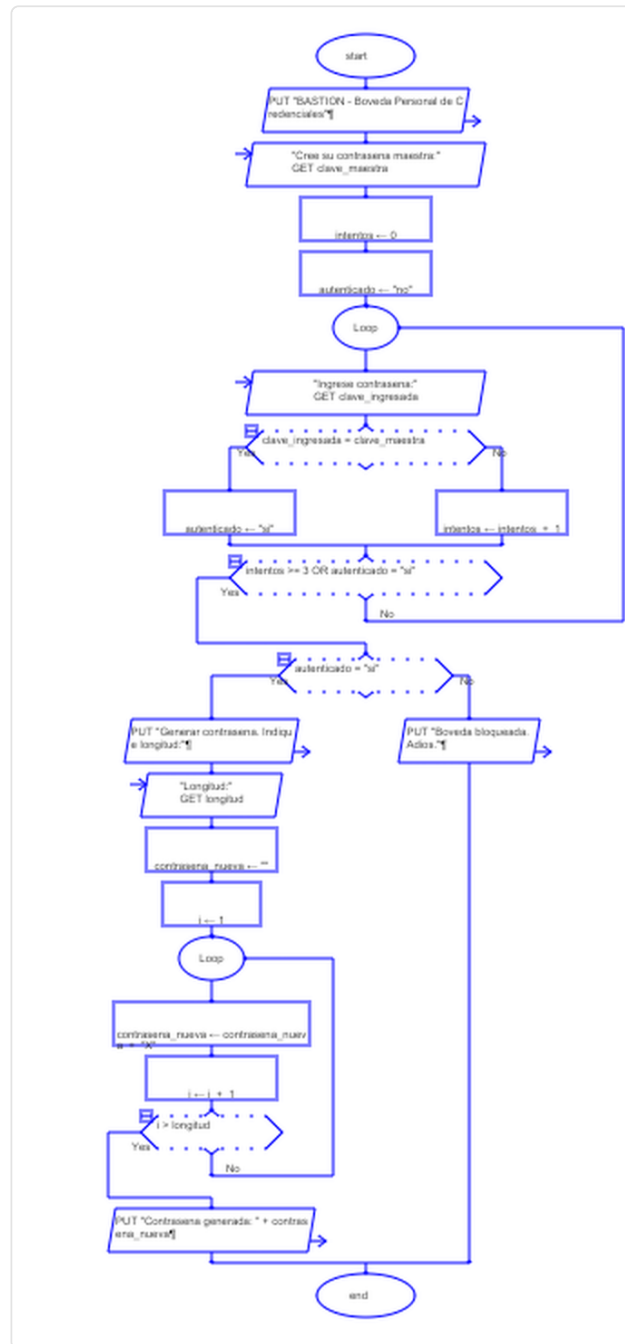


Figura 3. Diagrama de flujo del proceso principal elaborado en RAPTOR.

El sistema ejerce de forma explícita las estructuras repetitivas y de selección, junto con diccionarios, contadores y operadores relacionales y lógicos.

Bucle while con contador para el inicio de sesión

El control de acceso es el ejemplo más representativo. La función `iniciar_sesion()` emplea un contador `intentos` inicializado en cero y un bucle `while intentos < MAX_INTENTOS` (con `MAX_INTENTOS = 3`). En cada vuelta se compara el hash de lo ingresado con el hash almacenado: si coinciden, se concede el acceso y se retorna la bóveda; si no, el contador se incrementa con `intentos += 1` y se informa cuántos intentos restan. Si el bucle termina sin éxito, el programa muestra el aviso

de bloqueo y cierra con `sys.exit()`. Este mecanismo de limitación de intentos es coherente con la mitigación de ataques de adivinación en línea recomendada para verificadores de autenticación (Grassi et al., 2017).

Bucle for con range() para generar la contraseña

La construcción de la contraseña recorre tantas iteraciones como caracteres se deseen mediante `for i in range(longitud_deseada)`, agregando en cada paso un carácter elegido al azar del alfabeto. El alfabeto se arma previamente con condicionales que concatenan conjuntos de caracteres según lo elegido por el usuario.

Bucle for con .items() para buscar y listar

Tanto `buscar_credencial()` como `ver_credenciales()` recorren el diccionario de credenciales con `for servicio, datos in credenciales.items()`, lo que entrega simultáneamente la clave (nombre del servicio) y su valor (categoría, usuario y contraseña). En la búsqueda se compara `servicio.lower() == busqueda.lower()` para ignorar mayúsculas y minúsculas, y una bandera booleana registra si hubo coincidencia. En el listado, un contador `total += 1` totaliza las credenciales mostradas mientras la contraseña se presenta enmascarada.

Estructuras de datos: tupla, lista y diccionarios

BASTION integra los tres tipos de estructuras de datos estudiados. Las categorías permitidas se definen en una tupla inmutable, `CATEGORIAS`, ya que son valores fijos que no deben modificarse durante la ejecución; el menú las recorre con `enumerate` y se accede a la categoría elegida por su índice. El historial de accesos se guarda en una lista, `historial`, a la que la función `registrar_acceso` agrega un registro con la fecha y la hora en cada inicio de sesión. Los diccionarios son la estructura central del almacenamiento: las credenciales se guardan en un diccionario anidado donde la clave es el nombre del servicio y el valor es otro diccionario con `categoria`, `usuario` y `contrasena`. El uso del operador de pertenencia `in` sobre las claves permite detectar servicios duplicados de forma directa.

Cálculo de fortaleza con if / elif / else

La función `calcular_fortaleza()` clasifica la contraseña con una cadena `if / elif / else` que combina operadores relacionales y lógicos: longitud menor a 8 devuelve Débil; longitud hasta 11 devuelve Media; longitud de 12 o más junto con tres o más tipos de caracteres (`longitud >= 12 and tipos_seleccionados >= 3`) devuelve Fuerte; cualquier otro caso devuelve Media. El criterio prioriza la longitud como factor de robustez, en línea con las directrices que destacan la extensión del secreto memorizado por encima de la mera complejidad de composición (Grassi et al., 2017).

A continuación se ilustra el núcleo del generador, donde un bucle `for` con `range()` y `secrets.choice()` opera sobre un alfabeto construido con condicionales:

```
# Alfabeto base con minúsculas; se amplia según las opciones del usuario.
alfabeto = string.ascii_lowercase
if incluir_mayusculas:
    alfabeto += string.ascii_uppercase
if incluir_numeros:
    alfabeto += string.digits
if incluir_simbolos:
    alfabeto += "!@#$$%&*?-_ "

# Se arma la contraseña carácter por carácter.
contrasena_generada = ""
for i in range(longitud_deseada):
    contrasena_generada += secrets.choice(alfabeto)
```

4.4. Persistencia en JSON y hash SHA-256 de la clave maestra

La bóveda se persiste en un único archivo local, `boveda.json`, escrito por `guardar_boveda()` con `json.dump()` (con `indent=4` y `ensure_ascii=False` para conservar legibilidad y acentos) y leído por `cargar_boveda()` con `json.load()`. La estructura del archivo es un diccionario con tres claves: `hash_maestra`, que guarda el hash de la contraseña maestra; `credenciales`, que contiene el conjunto de servicios registrados; e `historial`, una lista con los registros de acceso.

La contraseña maestra nunca se almacena en claro. La función `calcular_hash()` codifica el texto a bytes UTF-8 y aplica `hashlib.sha256(...).hexdigest()`, produciendo una cadena hexadecimal de 64 caracteres. SHA-256 pertenece a la familia SHA-2 de funciones hash criptográficas aprobadas como estándar (Stallings, 2023). En el inicio de sesión y en el cambio de clave, el sistema vuelve a calcular el hash de lo ingresado y lo compara con el valor guardado, de modo que la verificación ocurre sin conservar el secreto original.

Aclaración de honestidad académica

Conviene precisar el alcance de seguridad de esta versión. En el estado actual, los campos de cada credencial, incluida la contraseña del servicio, se guardan en texto plano dentro del archivo JSON local; la única protección criptográfica aplicada recae sobre la contraseña maestra, resguardada mediante su hash SHA-256. Es decir, el archivo está protegido por la clave maestra en el sentido de control de acceso de la aplicación, pero su contenido no está cifrado en disco.

El diseño contempla, como evolución del producto, el cifrado autenticado AES-256-GCM para proteger la confidencialidad e integridad de las credenciales en reposo, junto con la derivación de la clave de cifrado a partir de la contraseña maestra mediante Argon2id, una función de derivación de clave resistente a ataques por hardware especializado. El modo GCM proporciona cifrado autenticado con datos asociados, combinando confidencialidad y verificación de integridad en una sola operación (Dworkin, 2007). Asimismo, la calidad de la aleatoriedad empleada en la generación de contraseñas es determinante para su seguridad; por ello el generador emplea el módulo `secrets` de la biblioteca estándar, que proporciona aleatoriedad criptográficamente segura apta para usos de seguridad, en lugar de un generador de propósito general, conforme a las consideraciones sobre aleatoriedad para

seguridad (Eastlake et al., 2005). El almacenamiento de contraseñas con funciones de hash o derivación adecuadas y el cifrado de los secretos en reposo responden, además, a las prácticas recomendadas en la materia (OWASP Foundation, 2025). Estas mejoras quedan planificadas y no forman parte de la entrega académica actual.

4.5. Implementación como aplicación web

Además de la versión de consola, el sistema se implementó como una aplicación web publicada y disponible para su prueba en línea, en la dirección <https://bluenovasas.github.io/bastion-uide/webapp/>. Esta versión refuerza el principio de la arquitectura en tres capas y, sobre todo, evidencia la integración real del código Python. La capa de presentación se construye con HTML, CSS y JavaScript, mientras que la capa de lógica es el mismo archivo `bastion.py`, ejecutado dentro del navegador mediante Pyodide, una distribución de Python compilada a WebAssembly. Cuando la persona usuaria genera una contraseña, crea la bóveda o calcula la fortaleza, la aplicación invoca las funciones reales de `bastion.py`, como `calcular_hash`, `calcular_fortaleza` y `enmascarar`, de modo que la lógica no se reescribe en JavaScript sino que se reutiliza el código del proyecto.

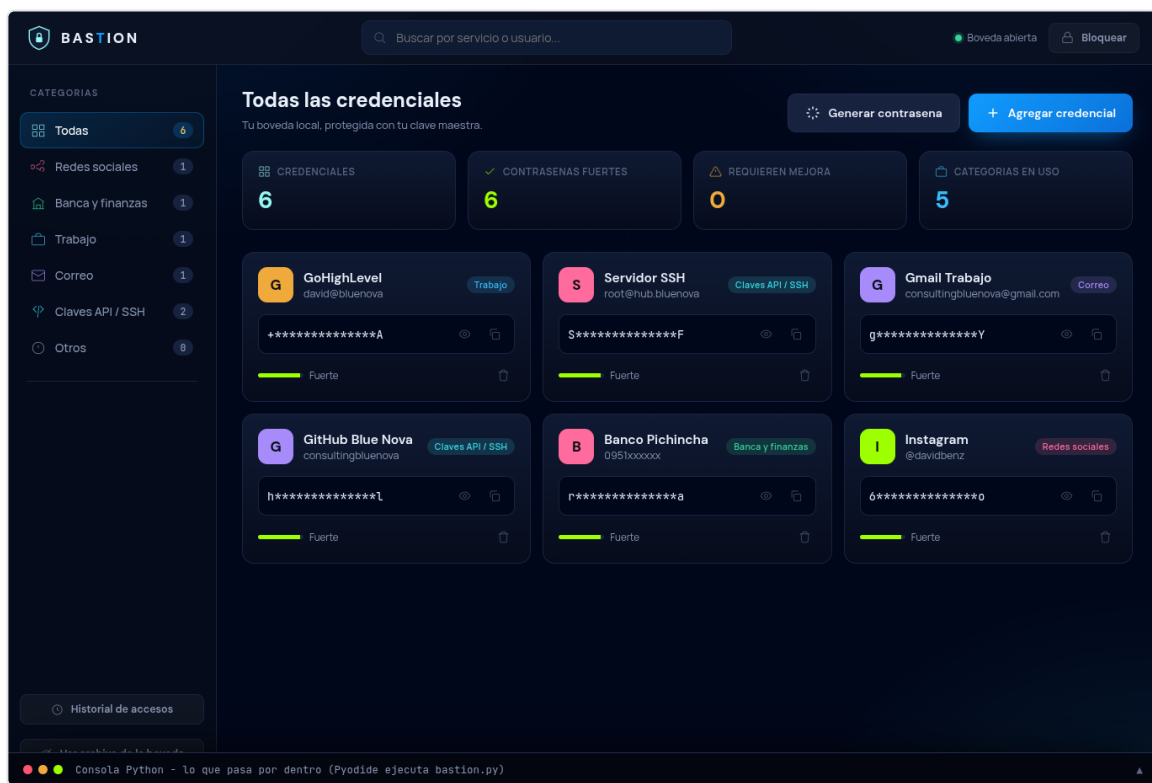


Figura 4. Aplicación web de BASTION: el dashboard ejecuta el código real de `bastion.py` mediante Pyodide.

La capa de persistencia, en esta versión, corresponde al almacenamiento local del navegador, donde la bóveda se conserva en formato JSON sin enviarse a ningún servidor, manteniendo el principio de operación local y sin nube. La aplicación incorpora un panel denominado "Consola Python" que muestra en tiempo real cada llamada a Python y su resultado, lo que hace transparente el funcionamiento interno del sistema y permite verificar que la lógica se ejecuta efectivamente en Python.

La aplicación se publicó mediante GitHub Pages, por lo que es accesible desde cualquier navegador sin instalar nada; la primera carga requiere conexión a internet para descargar el motor de Python. Esta implementación demuestra cómo una misma solución lógica puede ofrecerse a través de distintas interfaces sin alterar su núcleo, y acerca la herramienta a personas no técnicas, en línea con el objetivo de democratizar buenas prácticas de seguridad descrito en este trabajo.

5. Reflexión sobre el impacto de las nuevas tecnologías en la sociedad

La transformación digital ha modificado de manera profunda la forma en que las personas trabajan, se comunican y administran sus asuntos cotidianos. Actividades que antes dependían de la presencia física, como acceder al sistema de una entidad bancaria, coordinar un equipo de trabajo o gestionar la infraestructura de una empresa, hoy se resuelven mediante servicios en línea, plataformas colaborativas y entornos en la nube. Esta migración hacia lo digital ha traído consigo un fenómeno menos visible pero igualmente decisivo: la multiplicación constante de las credenciales que cada individuo debe administrar. Un profesional promedio mantiene accesos a correos institucionales y personales, redes sociales, plataformas de pago, repositorios de código, servidores remotos y servicios de mensajería, lo que genera decenas de combinaciones de usuario y contraseña. La consecuencia natural de esta sobrecarga es que muchas personas recurren a prácticas inseguras, como reutilizar la misma contraseña en múltiples servicios o elegir claves débiles y fáciles de recordar, exponiéndose a un riesgo que crece a la par de su huella digital.

En este escenario, los gestores de contraseñas y la ciberseguridad han dejado de ser asuntos exclusivos de especialistas para convertirse en componentes de la vida cotidiana. La seguridad de la información ya no se limita a proteger sistemas militares o corporativos, sino que protege la identidad, el patrimonio y la privacidad de cualquier ciudadano que utilice un dispositivo conectado. Un gestor de contraseñas responde a una necesidad social concreta: permite que las personas usen claves únicas, largas y de alta entropía sin la carga cognitiva de memorizarlas, trasladando esa responsabilidad a una bóveda protegida. De esta manera, la herramienta no solo resuelve un problema técnico, sino que democratiza el acceso a buenas prácticas de seguridad que de otro modo quedarían reservadas a quienes poseen conocimientos avanzados. Estándares de referencia como las directrices de identidad digital de NIST (Grassi et al., 2017) y las recomendaciones de almacenamiento seguro de contraseñas de OWASP (OWASP Foundation, 2025) reflejan precisamente este giro: la usabilidad y la protección deben coexistir para que la seguridad sea efectiva en la práctica.

Sin embargo, la adopción masiva de estas tecnologías revela una tensión permanente entre comodidad y seguridad o privacidad. Cada funcionalidad que simplifica la vida del usuario, como el inicio de sesión automático, la sincronización en la nube o el almacenamiento centralizado de todas las claves en un solo lugar, introduce a la vez un punto de concentración de riesgo. Una bóveda que reúne todas las credenciales de una persona se convierte, si no está debidamente protegida, en un objetivo de altísimo valor para un atacante. La comodidad puede inducir a delegar la seguridad sin comprender sus implicaciones, mientras que un exceso de fricción puede empujar al usuario de regreso a las prácticas inseguras que se buscaba evitar. El diseño responsable de cualquier solución informática debe, por tanto, equilibrar ambos extremos: ofrecer una experiencia fluida sin sacrificar la confidencialidad de los datos. BASTION asume esta tensión de manera explícita al operar de forma

local, sin depender de la nube, lo que reduce la superficie de ataque y mantiene al usuario en control directo de su información, a la vez que automatiza la generación de contraseñas robustas para no trasladarle esa carga.

La masificación del manejo de datos personales conlleva, además, una responsabilidad ética ineludible. Quien desarrolla o administra una herramienta que custodia información sensible asume un deber de cuidado frente a las personas cuyos datos protege. Este deber no es meramente técnico, sino moral y jurídico. La protección de datos personales se ha consolidado como un derecho que los marcos normativos reconocen y exigen respetar. En el caso ecuatoriano, la Ley Orgánica de Protección de Datos Personales (LOPD) establece principios como la finalidad, la minimización, la confidencialidad y la seguridad en el tratamiento de los datos, obligando a quienes los procesan a implementar medidas técnicas y organizativas adecuadas. Una solución como BASTION se alinea con este espíritu al limitar la recolección de información a lo estrictamente necesario, al evitar la transferencia de datos a terceros y al proteger la bóveda mediante mecanismos de autenticación. La ética del manejo de datos no se agota en cumplir la norma, sino en interiorizar que detrás de cada credencial almacenada existe una persona cuya privacidad y seguridad dependen de decisiones de diseño tomadas con rigor y honestidad.

Mirando hacia el futuro, las nuevas tecnologías anticipan una evolución del paradigma mismo de la autenticación. Las claves de acceso sin contraseña, conocidas como passkeys y sustentadas en el estándar FIDO2, proponen reemplazar la contraseña tradicional por pares de claves criptográficas que nunca se transmiten ni se almacenan en servidores, mitigando ataques de phishing y de reutilización. De forma paralela, el avance de la computación cuántica plantea desafíos a largo plazo para los algoritmos criptográficos actuales, lo que ha impulsado el desarrollo de la criptografía post-cuántica como línea de investigación para preservar la confidencialidad frente a capacidades de cómputo futuras. Conviene abordar estas proyecciones con mesura: las passkeys aún conviven con las contraseñas durante un periodo de transición, y la amenaza cuántica no representa un riesgo inmediato para la mayoría de los usuarios, sino un horizonte que la disciplina debe preparar con anticipación. Lejos de volver obsoletos a los gestores de contraseñas, estos avances refuerzan su papel como infraestructura de confianza capaz de incorporar nuevos mecanismos a medida que maduran.

En conjunto, estas reflexiones permiten situar a BASTION no como un ejercicio aislado de programación, sino como una respuesta concreta a una necesidad social genuina. La proliferación de credenciales, la centralidad de la ciberseguridad en la vida diaria, la tensión entre comodidad y protección, el deber ético frente a los datos personales y la evolución tecnológica del campo convergen en un mismo punto: las personas y las organizaciones necesitan herramientas accesibles, responsables y bien fundamentadas para custodiar su identidad digital. Al centralizar de forma local y segura credenciales técnicas dispersas, como accesos a servidores, claves de API y llaves SSH, y al construirse sobre los principios de estándares reconocidos, BASTION ilustra cómo una solución informática modesta en su alcance puede contribuir de manera significativa al bienestar y la autonomía de quienes la utilizan, demostrando que la tecnología adquiere su mayor valor cuando se pone al servicio de las necesidades reales de la sociedad.

El desarrollo de BASTION se organizó a lo largo de las ocho semanas que comprende la asignatura de Lógica de Programación, alineando cada avance del producto con las cuatro unidades del programa de estudios. Esta planificación incremental permitió que cada estructura de programación vista en clase encontrara una aplicación directa y verificable dentro del proyecto, de modo que el aprendizaje teórico se consolidara mediante la construcción progresiva de un artefacto de software real. El

enfoque por iteraciones, recomendado para proyectos académicos de alcance acotado (Sommerville, 2016), evitó la acumulación de trabajo al cierre y facilitó la detección temprana de errores en cada bloque funcional.

El cronograma se concibió como una secuencia de fases dependientes: las semanas 1 y 2 se destinaron al análisis del problema y al diseño previo (selección del software, diagramas de funcionalidad y arquitectura); las semanas 3 y 4 a la configuración del entorno de trabajo y del repositorio para el control de versiones, junto con los primeros avances de codificación; las semanas 5 y 6 a la implementación de las estructuras de decisión y de los bucles, con su correspondiente diagrama de flujo; y las semanas 7 y 8 al perfeccionamiento, la evaluación del código y la integración del producto final. La siguiente tabla detalla, semana a semana, la unidad y el tema correspondientes, la actividad concreta realizada sobre BASTION y el entregable asociado.

Semana	Unidad y tema	Actividad realizada en BASTION	Entregable
1	Unidad 1 - Temas 1 y 2: fundamentos de la programación, análisis y resolución de problemas	Identificación del problema (credenciales técnicas dispersas en Blue Nova Consulting), definición del alcance, selección del software a desarrollar (gestor de contraseñas CLI) y elección de Python con su biblioteca estándar	Documento de definición del problema y justificación de la propuesta
2	Unidad 1 - Temas 1 y 2: diagramas de funcionalidad y diagrama de arquitectura	Elaboración de los diagramas de funcionalidad (casos de uso CU-01 a CU-07 y flujo general) y del diagrama de arquitectura en tres capas (presentación, lógica de negocio y persistencia); diseño previo de la bóveda local en JSON con hash SHA-256	Diagramas de casos de uso, flujo y arquitectura; diseño conceptual del producto
3	Unidad 2 - Temas 3 y 4: inicio del desarrollo, configuración del entorno y de GitHub para el control de versiones	Instalación y configuración del entorno de Python, creación del repositorio en GitHub, definición de <code>.gitignore</code> y <code>LICENSE</code> , y establecimiento del flujo de confirmaciones para el control de versiones	Repositorio inicializado con estructura base y primer commit
4	Unidad 2 - Temas 3 y 4: variables y tipos de datos (str, int, bool), primeros avances de codificación	Creación del archivo <code>bastion.py</code> , declaración de variables y manejo de tipos básicos (cadenas para credenciales, enteros para contadores y booleanos para validaciones); codificación inicial de la lectura y escritura del archivo <code>boveda.json</code>	Esqueleto funcional del programa con persistencia básica en JSON
5	Unidad 3 - Temas 5 y 6: estructuras de decisión (if / elif / else), operadores relacionales y lógicos	Implementación de los menús de la interfaz CLI, las validaciones de entrada y el indicador de fortaleza (Débil, Media, Fuerte) mediante <code>if / elif / else</code> , operadores relacionales y operadores lógicos (<code>and</code> , <code>or</code> , <code>not</code> , <code>in</code>)	Módulo de menús y validaciones con estructuras de decisión operativas
6	Unidad 3 - Temas 5 y 6: estructuras repetitivas (while y for), contadores y comentarios del código	Programación del bucle <code>while</code> con contador de intentos para el inicio de sesión (máximo tres intentos y bloqueo), del bucle <code>for</code> con <code>range()</code> para generar la contraseña carácter por carácter y del bucle <code>for</code> con <code>.items()</code> para la búsqueda y el listado de credenciales; incorporación de contadores (<code>+= 1</code>) y comentarios explicativos; elaboración del diagrama de flujo en Raptor	Avance funcional del código con bucles y contadores; diagrama de flujo en Raptor
7	Unidad 4 - Temas 7 y 8: técnicas de programación funcional, evaluación y optimización del código	Refactorización del programa en funciones de propósito único (generador, validaciones, búsqueda, persistencia), evaluación de la legibilidad y la eficiencia, y participación en el foro mediante la revisión del código de los compañeros con retroalimentación constructiva	Código modularizado en funciones y aportes de evaluación en el foro
8	Unidad 4 - Temas 7 y 8: estructuras de datos (listas, tuplas, diccionarios) y	Integración de todos los módulos, consolidación del uso de diccionarios para categorías y almacenamiento de credenciales, pruebas finales de los casos de uso, documentación en el <code>README.md</code> y entrega del producto terminado	Producto final integrado, documentado y publicado en el repositorio

Semana	Unidad y tema	Actividad realizada en BASTION	Entregable
	funciones, integración del producto final		

La distribución presentada evidencia una correspondencia directa entre la teoría de cada unidad y su materialización en el código. Las decisiones de diseño adoptadas desde la primera unidad, como el almacenamiento local del hash de la contraseña maestra y la separación en tres capas, se mantuvieron coherentes durante toda la construcción, lo que reduce la superficie de ataque del producto y favorece su mantenibilidad (Pressman y Maxim, 2021). De este modo, el cronograma no solo organizó el tiempo disponible, sino que garantizó que el aprendizaje de las estructuras lógicas se tradujera en un gestor de credenciales funcional y verificable al cierre del periodo académico.

7.1 Conclusiones

El desarrollo de BASTION, bóveda personal de credenciales, permite extraer un conjunto de conclusiones que conectan de manera coherente los objetivos planteados, las estructuras de programación empleadas y el marco de estándares que orienta el diseño.

- **Las estructuras fundamentales de programación bastan para construir un sistema funcional y útil.** El proyecto demuestra que con bucles `while` (control de intentos de inicio de sesión), bucles `for` con `range()` (generación carácter por carácter de la contraseña), iteraciones con `.items()` (búsqueda y listado de credenciales), estructuras `if/elif/else` (menús, validaciones y cálculo de fortaleza), diccionarios (categorías y almacenamiento) y contadores con incremento `+= 1`, es posible resolver un problema real de gestión de credenciales. La lógica de programación no es un ejercicio abstracto, sino el medio para materializar funcionalidades concretas y verificables.
- **La separación en tres capas favorece la claridad y el mantenimiento.** Al distribuir la responsabilidad entre presentación (menús CLI), lógica de negocio (generador, validaciones y búsqueda) y persistencia (lectura y escritura del archivo JSON), el sistema gana orden interno y se vuelve más fácil de comprender, probar y ampliar. Esta organización modular es coherente con los principios de diseño descritos por Sommerville (2016) y Pressman y Maxim (2021), quienes destacan que la descomposición en componentes con responsabilidades bien delimitadas reduce la complejidad y eleva la calidad del software.
- **La decisión de almacenar solo el hash de la contraseña maestra refleja una buena práctica de seguridad reconocida.** El sistema nunca guarda la contraseña maestra en texto claro, sino su huella calculada con SHA-256, lo que evita exponer el secreto que protege la bóveda. Esta orientación es consistente con las recomendaciones de OWASP Foundation (2025) sobre almacenamiento seguro de contraseñas y con el enfoque de verificación de identidad de Grassi et al. (2017) en NIST SP 800-63B, que privilegian no conservar secretos recuperables.
- **El carácter local del almacenamiento reduce de forma deliberada la superficie de ataque.** Al prescindir de la nube y mantener la bóveda en un archivo JSON en el propio equipo, el sistema elimina toda la categoría de riesgos asociada a servicios remotos, transmisión por red y dependencia de terceros. Esta elección de diseño, alineada con el principio de minimizar la

exposición descrito por Stallings (2023), aporta una ventaja de seguridad concreta para el contexto de uso individual previsto.

- **El proyecto consolida competencias de programación y de seguridad de manera integrada.** BASTION no solo aplica con corrección las estructuras de control y de datos exigidas por la asignatura, sino que lo hace dentro de un dominio de ciberseguridad fundamentado en estándares de referencia, lo que demuestra la capacidad de traducir requisitos de seguridad en lógica de programación concreta y de justificar cada decisión técnica con base en buenas prácticas reconocidas.
- **La solución se llevó más allá del aula con una aplicación web funcional.** El mismo código de `bastion.py` se ejecuta en el navegador mediante Pyodide y se publicó en línea, lo que demuestra que la lógica desarrollada es reutilizable a través de distintas interfaces y resulta accesible para personas no técnicas. Un panel de consola hace visible, en tiempo real, la ejecución del código Python, lo que refuerza la transparencia del sistema.

7.2 Implicaciones

BASTION aporta una herramienta operativa que centraliza credenciales técnicas que, de otro modo, tienden a quedar dispersas en notas, archivos sueltos o en la memoria del usuario. El sistema genera contraseñas robustas mediante un proceso controlado, las clasifica por categorías (Redes sociales, Banca y finanzas, Trabajo, Correo, Claves API/SSH y Otros) y las protege detrás de una única contraseña maestra, lo que disminuye la carga cognitiva de recordar múltiples secretos y desincentiva la reutilización de contraseñas débiles.

El principal beneficiario es el autor en su rol de responsable técnico de Blue Nova Consulting, agencia ecuatoriana en la que conviven accesos a servidores, claves de API y llaves SSH que requieren un punto único y ordenado de consulta. Al centralizar esos accesos en una bóveda local, el sistema reduce el riesgo de extravío de credenciales críticas y agiliza las tareas de administración cotidiana.

En un plano más amplio, el proyecto beneficia a usuarios individuales y a profesionales técnicos que necesitan una solución sencilla, sin dependencia de la nube y sin costo de licenciamiento, así como al propio proceso de aprendizaje académico: demuestra que conceptos de lógica de programación pueden articularse en un producto con valor práctico y con criterios de seguridad explícitos. El indicador de fortaleza (Débil, Media, Fuerte) cumple además una función formativa, pues retroalimenta al usuario sobre la calidad de las contraseñas y promueve mejores hábitos de seguridad.

7.3 Limitaciones

Por honestidad académica, conviene reconocer con precisión los límites del alcance actual del producto, que corresponde a un producto mínimo viable con fines didácticos.

- **Cifrado de las credenciales no implementado en la versión académica.** Aunque la contraseña maestra se protege con SHA-256, las credenciales almacenadas en el archivo JSON residen sin un cifrado simétrico real. La versión vigente contempla AES-256-GCM y la derivación de clave con Argon2id como evolución del producto, pero estos mecanismos aún no operan, por lo

que la confidencialidad del contenido de la bóveda frente a un acceso directo al archivo es todavía limitada.

- **Almacenamiento local sin soporte multiusuario.** La bóveda está diseñada para un único propietario en un solo equipo. No existe gestión de cuentas separadas, control de permisos ni mecanismos de acceso concurrente, lo que restringe su aplicación a escenarios individuales y deja fuera el uso compartido dentro de un equipo de trabajo.
- **Ausencia de segundo factor de autenticación.** El acceso depende exclusivamente de la contraseña maestra. No se ha incorporado aún un segundo factor, de modo que la seguridad del sistema descansa por completo en la fortaleza y el resguardo de ese único secreto.
- **Núcleo basado en la biblioteca estándar.** La versión académica emplea únicamente módulos estándar de Python (`hashlib`, `json`, `os`, `secrets`, `string`, `getpass`, `sys`, `datetime`) en su lógica. Aunque se desarrolló una interfaz web que complementa la versión de consola, aún no se incorporan bibliotecas criptográficas dedicadas ni respaldo automatizado, lo que mantiene el producto en un nivel adecuado para la demostración de competencias pero pendiente de endurecimiento para un despliegue profesional.

Proyección para superar las limitaciones

Estas limitaciones definen una hoja de ruta clara de evolución. En materia de confidencialidad, se prevé cifrar el contenido de la bóveda con AES-256-GCM, modo de operación autenticado que protege a la vez la confidencialidad y la integridad de los datos (Dworkin, 2007, NIST SP 800-38D), y derivar la clave de cifrado a partir de la contraseña maestra mediante Argon2id, fortaleciendo la resistencia frente a ataques de fuerza bruta. La generación de contraseñas ya emplea aleatoriedad criptográficamente segura mediante el módulo `secrets`, en línea con las recomendaciones de Eastlake et al. (2005) en la RFC 4086. Asimismo, se proyecta incorporar un segundo factor de autenticación y evaluar esquemas de respaldo cifrado, conforme a los lineamientos de verificación de identidad de Grassi et al. (2017). El soporte multiusuario y una eventual interfaz gráfica quedan planteados como extensiones posteriores, manteniendo la arquitectura en tres capas como base que facilita estas ampliaciones sin reescribir el sistema.

Referencias

-
- Dworkin, M. (2007). *Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC* (NIST Special Publication 800-38D). National Institute of Standards and Technology.
- Eastlake, D., Schiller, J., & Crocker, S. (2005). *Randomness requirements for security* (RFC 4086). Internet Engineering Task Force.
- Grassi, P. A., Garcia, M. E., & Fenton, J. L. (2017). *Digital identity guidelines: Authentication and lifecycle management* (NIST Special Publication 800-63B). National Institute of Standards and Technology.
- Object Management Group. (2017). *Unified Modeling Language (UML), version 2.5.1*. OMG.
- OWASP Foundation. (2025). *Password storage cheat sheet*. OWASP Cheat Sheet Series.
- Pressman, R. S., & Maxim, B. R. (2021). *Software engineering: A practitioner's approach* (9.a ed.). McGraw-Hill Education.
- Sommerville, I. (2016). *Software engineering* (10.a ed.). Pearson Education.

Stallings, W. (2023). *Cryptography and network security: Principles and practice* (8.a ed.). Pearson Education.